

Проектная работа

**«Библиотека построения гистограмм
на основе потоковых данных»**

Функциональные требования

1. Гистограмма должна строиться на основе потока данных (данных поступающих в реальном времени)
2. Данные поступающие в реальном времени должны быть многомерными
3. Границы гистограммы должны варьироваться динамически в зависимости от поступающих данных
4. Количество бинов должно определяться статически или задаваться в реальном времени пользователем
5. В библиотеке должен присутствовать функционал расчёта и отображения ковариации и среднего

Нефункциональные требования

1. Библиотека должна быть написана на языке `kotlin`
2. Процесс сбора и подготовки данных для гистограммы должен происходить без блокировки процесса построения гистограммы
3. Библиотека должна иметь возможность хранить редукцию данных за промежуток времени

Опциональные требования

1. Визуализация гистограммы при помощи любой GUI библиотеки.
2. Построение гистограммы в 3d пространстве
3. Дополнительно с цветовой палитрой
4. Сделать библиотеку универсальной для возможности дальнейшего расширения под произвольные форматы данных

Определение

Гистограмма — это непараметрическая оценка функции плотности вероятности (Probability Density Function, PDF) непрерывной случайной величины по конечной выборке. Это аппроксимация, строящаяся путем группировки данных в интервалы (бины, англ. bins) и нормировки частоты попаданий в каждый интервал на его ширину и объем выборки.

!Важно — гистограмма отображает распределение **непрерывных** данных, а не количественные метрики для **категориальных** данных.

Решаемые в работе проблемы

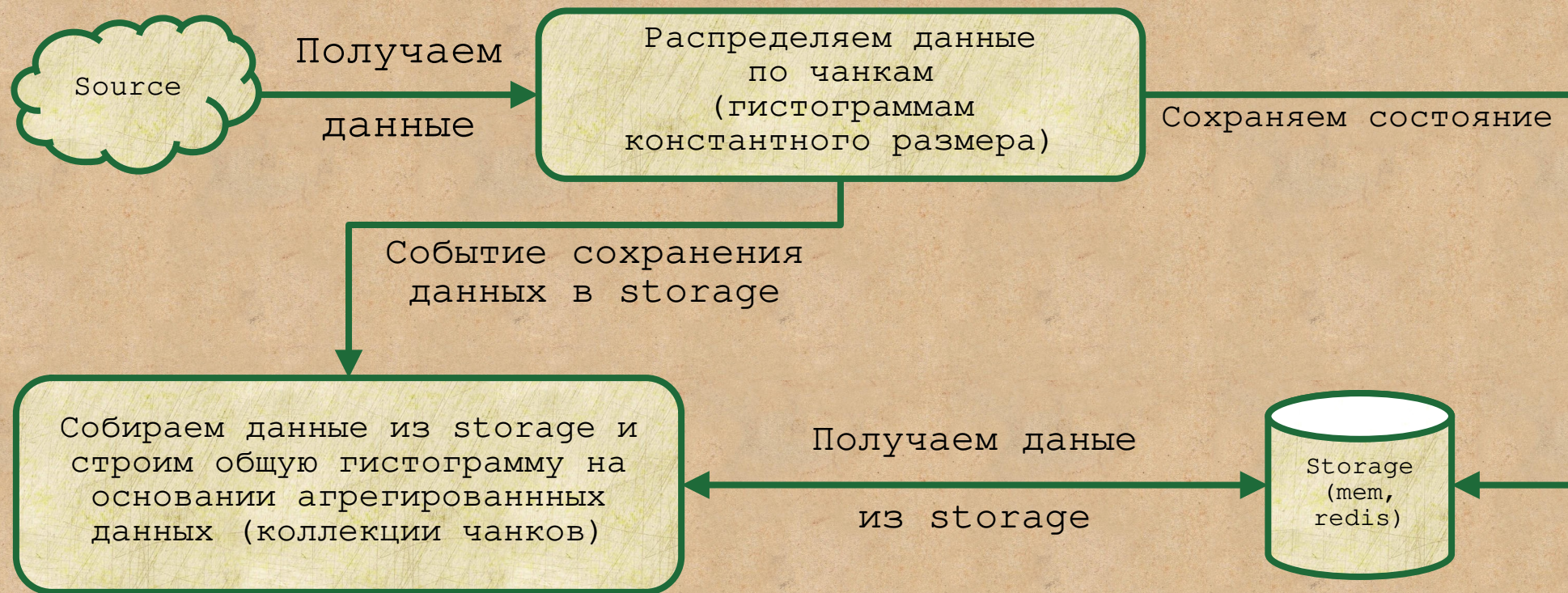
Проблема дискретности данных

Потоковые данные могут оказаться дискретными, что затруднит расчёт функции плотности вероятности и построение гистограммы.

Проблема оптимального разбиения гистограммы

Проблему оптимального разбиения гистограммы в области отсутствия границ данных и возможной дискретности данных мы рассмотрим отдельно.

Предлагаемое решение



Алгоритм построения чанка

1. Из потока данных получаем "точку" (изолированную область данных)
 1. Определяем chunk для записи точки
 2. Если подходящей области не обнаружено, то создаем её. Создание области необходимо будет осуществлять вокруг точки согласно принятой константе.

Пример:

1. Получаем и обрабатываем точку [1,1,2]
 1. Определяем, есть ли уже область подходящая для точки
 2. Если точка есть, то добавляем её к гистограмме
 3. Если точки нет, то создаем область размером 10x10x10 с координатами $P_{\min}(0,0,0)$, $P_{\max}(10,10,10)$. Количество бинов определяется статически
 4. Добавляем точку к нужному бину
 5. После достижения чанком определённого размера, сохраняем построенную гистограмму в storage и отправляем событие в сервис построения общей гистограммы

Алгоритм построения гистограммы

1. Создаем область для гистограммы. Определяется статически, например как для периода просмотра котировок акций. Это может происходить по запросу пользователя или статически на этапе MVP.
2. Получаем событие о сохранении чанка в **storage**
3. Идём в **storage** за данными на основании полученного события
4. Распределяем данные из полученного чанка по бинам гистограммы
 1. Если какой-то бин чанка попадает на границу бина гистограммы, то данные из него делятся пропорционально (уходят в какой-либо из бинов на этапе MVP) .

Алгоритм определения количества бинов

1.Т.к. мы не можем угадать границы потоковых данных, то предлагается выбирать количество бинов по формуле Стёрджеса.

2.Количество интервалов n определяется как:

$$n = 1 + \log_2(N)$$

3.Но, т.к. при потоковых данных мы не знаем N на момент построения гистограммы, то предлагается взять за N границы значений гистограммы.

Анализ исходников

В ходе проведения работы были проанализированы 3 библиотеки для построения гистограмм. А именно:

1.Cern.root (c++)

2.Numpy (python)

3.Plotters (rust)

Для каждой был проведён небольшой нагрузочный тест.

По результатам анализа было принято решение использовать для непосредственно построения объекта гистограммы ориентироваться на алгоритмы `cern.root`, т.к. они наиболее приближены к условиям обозначенным в алгоритмах ранее, а также показали наиболее стабильный и быстрый вариант построения объекта гистограммы.

Замечания с первой презентации

Определять количество чанков на основании границы общей гистограммы, определённой клиентом (пользователем) .

Подумать о различной размерности данных.

Результаты промежуточного этапа

1. Сделан функционал для демонстрации работы гистограмматора потоковых данных для одномерной целочисленной гистограммы.

1. Разработана система интерфейсов

2. Созданы дефолтные реализации данных интерфейсов

3. Для генератора потоковых данных и для конфигуратора гистограммы сделаны реализации для целочисленных значений

2. Сделано десктопное приложение для демонстрации распределения весов по бинам в общей гистограмме.

3. Записано демонстрационное видео, которое можно скачать по ссылке: https://disk.yandex.ru/i/9Ri_aM3zx-cmfg (144МБ)

4. Код выложен на github. С ним можно ознакомиться по ссылке: <https://github.com/Gurikin/flow-histogram>

5. Документация по api и архитектуре лежит в папке docs